

Determinación de circuitos en un grafo (C++)

Generated by Doxygen 1.16.1

1 Determinación de circuitos en un grafo (C++)	1
1.1 Características	1
1.2 Estructura del proyecto	1
1.3 Documentación	2
1.4 Ejecución	2
1.5 Autora	2
1.6 Maestro	2
1.7 Materia y Programa Académico	2
1.8 Fecha	2
2 Análisis	3
2.1 Cuestiones	3
2.2 Análisis técnico	3
2.3 Justificación teórica	3
3 Pruebas	5
3.1 Resultados de Pruebas	5
4 Bibliografía	15
5 Directory Hierarchy	17
5.1 Directories	17
6 Class Index	19
6.1 Class List	19
7 File Index	21
7.1 File List	21
8 Directory Documentation	23
8.1 headers Directory Reference	23
8.2 src Directory Reference	23
9 Class Documentation	25
9.1 Grafo< T > Class Template Reference	25
9.1.1 Detailed Description	25
9.1.2 Constructor & Destructor Documentation	25
9.1.2.1 Grafo()	25
9.1.3 Member Function Documentation	26
9.1.3.1 agregarArista()	26
9.1.3.2 dijkstra()	26
10 File Documentation	29
10.1 headers/dijkstra.h File Reference	29
10.1.1 Detailed Description	29
10.2 dijkstra.h	30

10.3 README.md File Reference	30
10.4 src/dijkstra.cpp File Reference	30
10.4.1 Detailed Description	30
10.5 dijkstra.cpp	30
10.6 src/main.cpp File Reference	31
10.6.1 Detailed Description	32
10.6.2 Function Documentation	32
10.6.2.1 main()	32
10.6.2.2 menu()	33
10.7 main.cpp	34
Index	37

Chapter 1

Determinación de circuitos en un grafo (C++)

Implementación de una clase [Grafo](#) para modelar redes de datos y encontrar los caminos más cortos entre sus vertices, sea el grafo dirigido o no dirigido, haciendo uso del algoritmo de Dijkstra.

1.1 Características

- Menú interactivo.
- Inicialización de grafo con n vértices.
- Uso de una matriz de adyacencia.
- Inserción de aristas.
- Visualización del camino más corto desde un vértice de inicio a los demás vértices del grafo.

1.2 Estructura del proyecto

- doc/
 - html/
 - latex/
- images/
 - Imágenes de resultados de pruebas.
- src/
 - [dijkstra.cpp](#)
 - [main.cpp](#)
- headers/
 - [dijkstra.h](#)

1.3 Documentación

Toda la documentación ha sido generada con Doxygen.

Para visualizar: Dirijase a `doc/html/index.html` Posteriormente, abra con su navegador de preferencia.

Para visualizar todos los comentarios: Vaya a "Class Members", después seleccione "All".

O abra el archivo PDF "refman.pdf".

1.4 Ejecución

- Usando comandos: Abra su terminal, y ejecute el siguiente código...

```
g++ src/main.cpp -lheaders -o dijkstra ./dijkstra
```

1.5 Autora

Camila Andrea Jaimes Martinez.

1.6 Maestro

Manuel Balderas Victoria.

1.7 Materia y Programa Académico

- Estructura de Datos (Aula).
- Ingeniería en Software y Minería de Datos (1er año).

1.8 Fecha

Viernes 7 de Abril, 2026.

Chapter 2

Análisis

Análisis técnico y Justificación teórica del programa.

2.1 Cuestiones

En este apartado se responden las siguientes cuestiones...

- 1) Análisis de la complejidad algorítmica de su implementación ($O(V^2)$ o $O(E \log V)$).
- 2) Justificación teórica: ¿Por qué una búsqueda en anchura (BFS) estándar no es capaz de encontrar la ruta más corta en un grafo donde las aristas tienen diferentes pesos? Explique cómo Dijkstra soluciona este problema.

2.2 Análisis técnico

1) La complejidad algorítmica de la implementación del algoritmo de Dijkstra, al utilizarse una matriz de adyacencia, es de $O(V^2)$. Su complejidad se debe a que, primero, en la función para determinar la distancia mínima (distanciaMin), se recorren todos los vértices hasta encontrar el vértice no visitado con la distancia mínima, lo cual lo categoriza como $O(V)$.

Posteriormente, en la función correspondiente al algoritmo de Dijkstra (dijkstra), se itera nuevamente sobre los vértices del grafo para actualizar las distancias de sus vértices adyacentes y encontrar el vértice no visitado con la distancia mínima, lo cual también lo categoriza como $O(V)$.

Por lo tanto, ambas funciones en conjunto dan como resultado una complejidad $O(V) * O(V) = O(V^2)$.

2.3 Justificación teórica

2) Una búsqueda en anchura (BFS) estándar es un algoritmo que puede recorrer grafos no ponderados (cuyas aristas no tienen costo o peso), encontrando el camino mas corto en cuestión de aristas. Logra su cometido utilizando una cola para acceder a los vertices adyacentes desde un vértice de inicio, es decir, recorre el grafo por niveles.

Debido a lo anterior, si se aplicara el algoritmo de BFS en un grafo ponderado, no tomaría en cuenta los pesos asignados a las aristas entre los vértices, haciéndolo susceptible a encontrar caminos más cortos en cuestión de aristas que de pesos.

Ante esta situación, el algoritmo de Dijkstra es la mejor opción para encontrar el camino más corto en un grafo ponderado, sea dirigido o no, ya que calcula la distancia mínima desde un vertice de inicio a los demás vértices utilizando una matriz de adyacencia para registrar los pesos de las aristas entre los vértices y actualizar las distancias acumuladas a través de los vértices adyacentes.

Chapter 3

Pruebas

Ejecución del programa.

3.1 Resultados de Pruebas

Las siguientes imágenes muestran la ejecución exitosa del programa correspondiente al algoritmo de Dijkstra, el cual tiene las funciones de...

- Inicialización de grafo con n vértices.
- Uso de una matriz de adyacencia.
- Inserción de aristas.
- Visualización del camino más corto desde un vértice de inicio a los demás vértices del grafo.

El primer grafo de ejemplo es un grafo no dirigido de 5 vértices, los cuales son del 0 al 4 debido a la indexación de los vértices. A continuación, se muestra la ejecución del programa para este grafo.

```

camila@MacBook-Air-de-Camila Desktop % cd alg_dijkstra
camila@MacBook-Air-de-Camila alg_dijkstra % ./dijkstra

----- i MENÚ ! -----
1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 1

¿De cuántos vértices desea crear el grafo?: 5

¡El grafo se ha creado exitosamente con 5 vértices!

----- i MENÚ ! -----
1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 2

Ingrese el vértice de inicio: 0

Ingrese el vértice de destino: 1

Ingrese el peso de la arista: 5

¿Es un grafo dirigido? (1 para sí, 0 para no): 0

¡La arista se ha creado exitosamente!

----- i MENÚ ! -----
1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 2

Ingrese el vértice de inicio: 0

Ingrese el vértice de destino: 3

Ingrese el peso de la arista: 3

¿Es un grafo dirigido? (1 para sí, 0 para no): 0

¡La arista se ha creado exitosamente!

----- i MENÚ ! -----
1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

```

Figure 3.1 Ejecución del programa en el grafo no dirigido (Parte 1)

```
----- i MENÚ ! -----
1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 0

----- i MENÚ ! -----
1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 2

Ingrese el vértice de inicio: 0

Ingrese el vértice de destino: 2

Ingrese el peso de la arista: 7

¿Es un grafo dirigido? (1 para sí, 0 para no): 0

¡La arista se ha creado exitosamente!

----- i MENÚ ! -----
1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 2

Ingrese el vértice de inicio: 2

Ingrese el vértice de destino: 3

Ingrese el peso de la arista: 1

¿Es un grafo dirigido? (1 para sí, 0 para no): 0

¡La arista se ha creado exitosamente!

----- i MENÚ ! -----
1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 2

Ingrese el vértice de inicio: 2

Ingrese el vértice de destino: 4

Ingrese el peso de la arista: 10
```

Figure 3.2 Ejecución del programa en el grafo no dirigido (Parte 2)

```

3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 2

Ingrese el vértice de inicio: 2

Ingrese el vértice de destino: 4

Ingrese el peso de la arista: 10

¿Es un grafo dirigido? (1 para sí, 0 para no): 0

¡La arista se ha creado exitosamente!

----- i MENÚ ! -----

1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 2

Ingrese el vértice de inicio: 4

Ingrese el vértice de destino: 3

Ingrese el peso de la arista: 6

¿Es un grafo dirigido? (1 para sí, 0 para no): 0

¡La arista se ha creado exitosamente!

----- i MENÚ ! -----

1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 3

¿Cuál es el vértice de inicio? (Implementación del algoritmo de Dijkstra): 0
----- Distancias desde el vértice 0 -----

--> Para llegar al vertice 0 : 0
--> Para llegar al vertice 1 : 5
--> Para llegar al vertice 2 : 4
--> Para llegar al vertice 3 : 3
--> Para llegar al vertice 4 : 9

----- i MENÚ ! -----

1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 4

camila@MacBook-Air-de-Camila alg_dijkstra %

```

Figure 3.3 Ejecución del programa en el grafo no dirigido (Parte 3)

El segundo grafo de ejemplo es un grafo dirigido de 8 vértices, los cuales son del 0 al 7 debido a la indexación de los vértices. A continuación, se muestra la ejecución del programa para este grafo.

```
|camila@MacBook-Air-de-Camila alg_dijkstra % ./dijkstra

----- | MENÚ ! -----
1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 1

¿De cuántos vértices desea crear el grafo?: 8

¡El grafo se ha creado exitosamente con 8 vértices!

----- | MENÚ ! -----
1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 2

Ingrese el vértice de inicio: 0

Ingrese el vértice de destino: 6

Ingrese el peso de la arista: 11

¿Es un grafo dirigido? (1 para sí, 0 para no): 1

¡La arista se ha creado exitosamente!

----- | MENÚ ! -----
1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 2

Ingrese el vértice de inicio: 0

Ingrese el vértice de destino: 3

Ingrese el peso de la arista: 6

¿Es un grafo dirigido? (1 para sí, 0 para no): 1

¡La arista se ha creado exitosamente!

----- | MENÚ ! -----
1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 2
```

Figure 3.4 Ejecución del programa en el grafo dirigido (Parte 1)

```

--> Seleccione una opción: 2

Ingrese el vértice de inicio: 3

Ingrese el vértice de destino: 2

Ingrese el peso de la arista: 2

¿Es un grafo dirigido? (1 para sí, 0 para no): 1

¡La arista se ha creado exitosamente!

----- i MENÚ ! -----

1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 2

Ingrese el vértice de inicio: 2

Ingrese el vértice de destino: 3

Ingrese el peso de la arista: 5

¿Es un grafo dirigido? (1 para sí, 0 para no): 1

¡La arista se ha creado exitosamente!

----- i MENÚ ! -----

1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 2

Ingrese el vértice de inicio: 2

Ingrese el vértice de destino: 5

Ingrese el peso de la arista: 8

¿Es un grafo dirigido? (1 para sí, 0 para no): 1

¡La arista se ha creado exitosamente!

----- i MENÚ ! -----

1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 2

Ingrese el vértice de inicio: 2

Ingrese el vértice de destino: 4

```

Figure 3.5 Ejecución del programa en el grafo dirigido (Parte 2)

```

Ingrese el vértice de destino: 4
Ingrese el peso de la arista: 4
¿Es un grafo dirigido? (1 para sí, 0 para no): 1
¡La arista se ha creado exitosamente!

----- i MENÚ ! -----
1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 2
Ingrese el vértice de inicio: 4
Ingrese el vértice de destino: 3
Ingrese el peso de la arista: 9
¿Es un grafo dirigido? (1 para sí, 0 para no): 1
¡La arista se ha creado exitosamente!

----- i MENÚ ! -----
1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 2
Ingrese el vértice de inicio: 4
Ingrese el vértice de destino: 6
Ingrese el peso de la arista: 7
¿Es un grafo dirigido? (1 para sí, 0 para no): 1
¡La arista se ha creado exitosamente!

----- i MENÚ ! -----
1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 2
Ingrese el vértice de inicio: 4
Ingrese el vértice de destino: 7
Ingrese el peso de la arista: 5
¿Es un grafo dirigido? (1 para sí, 0 para no): 1

```

Figure 3.6 Ejecución del programa en el grafo dirigido (Parte 3)

```

Ingrese el peso de la arista: 5

¿Es un grafo dirigido? (1 para sí, 0 para no): 1

¡La arista se ha creado exitosamente!

----- ¡ MENÚ ! -----
1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 2

Ingrese el vértice de inicio: 7

Ingrese el vértice de destino: 4

Ingrese el peso de la arista: 10

¿Es un grafo dirigido? (1 para sí, 0 para no): 1

¡La arista se ha creado exitosamente!

----- ¡ MENÚ ! -----
1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 2

Ingrese el vértice de inicio: 5

Ingrese el vértice de destino: 7

Ingrese el peso de la arista: 3

¿Es un grafo dirigido? (1 para sí, 0 para no): 1

¡La arista se ha creado exitosamente!

----- ¡ MENÚ ! -----
1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 2

Ingrese el vértice de inicio: 7

Ingrese el vértice de destino: 1

Ingrese el peso de la arista: 1

¿Es un grafo dirigido? (1 para sí, 0 para no): 1

¡La arista se ha creado exitosamente!

```

Figure 3.7 Ejecución del programa en el grafo dirigido (Parte 4)


```

Ingrese el vértice de inicio: 7
Ingrese el vértice de destino: 1
Ingrese el peso de la arista: 1
¿Es un grafo dirigido? (1 para sí, 0 para no): 1
¡La arista se ha creado exitosamente!

----- i MENÚ ! -----
1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 2

Ingrese el vértice de inicio: 1
Ingrese el vértice de destino: 5
Ingrese el peso de la arista: 9
¿Es un grafo dirigido? (1 para sí, 0 para no): 1
¡La arista se ha creado exitosamente!

----- i MENÚ ! -----
1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 3

¿Cuál es el vértice de inicio? (Implementación del algoritmo de Dijkstra): 0
----- Distancias desde el vértice 0 -----

--> Para llegar al vertice 0 : 0
--> Para llegar al vertice 1 : 18
--> Para llegar al vertice 2 : 8
--> Para llegar al vertice 3 : 6
--> Para llegar al vertice 4 : 12
--> Para llegar al vertice 5 : 16
--> Para llegar al vertice 6 : 11
--> Para llegar al vertice 7 : 17

----- i MENÚ ! -----
1) Crear un grafo de n vértices.
2) Agregar aristas ponderadas.
3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
4) Salir del programa.

--> Seleccione una opción: 4

Saliendo del programa...
camila@MacBook-Air-de-Camila alg_dijkstra %

```

Figure 3.8 Ejecución del programa en el grafo dirigido (Parte 5)

Chapter 4

Bibliografía

Fuentes consultadas.

Las fuentes consultadas (en formato APA) para la realización de este programa fueron...

- 1) Breadth first search or BFS for a graph. (2012, marzo 20). GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/>
- 2) C++ vector library. (s/f). W3schools.com. Recuperado el 11 de mayo de 2026, de https://www.w3schools.com/cpp/cpp_ref_vector.asp
- 3) Dijkstra's algorithm. (2012, noviembre 25). GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/dijkstras-shortest-path-algorithm-greedy-algo-7/>
- 4) Introduction to graph data structure. (2020, julio 15). GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/introduction-to-graphs-data-structure-and-algorithm-tutorials/>

Chapter 5

Directory Hierarchy

5.1 Directories

headers	23
dijkstra.h	29
src	23
dijkstra.cpp	30
main.cpp	31

Chapter 6

Class Index

6.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

[Grafo< T >](#) 25

Chapter 7

File Index

7.1 File List

Here is a list of all files with brief descriptions:

headers/ dijkstra.h	
Se define la estructura de un grafo pesado para la implementación del algoritmo de Dijkstra . . .	29
src/ dijkstra.cpp	
Implementación de de los constructores y métodos de la plantilla Grafo	30
src/ main.cpp	
El programa completo de la clase Grafo y el algoritmo de Dijkstra	31

Chapter 8

Directory Documentation

8.1 headers Directory Reference

Files

- file [dijkstra.h](#)

Se define la estructura de un grafo pesado para la implementación del algoritmo de Dijkstra.

8.2 src Directory Reference

Files

- file [dijkstra.cpp](#)

Implementación de de los constructores y métodos de la plantilla [Grafo](#).

- file [main.cpp](#)

El programa completo de la clase [Grafo](#) y el algoritmo de Dijkstra.

Chapter 9

Class Documentation

9.1 Grafo< T > Class Template Reference

```
#include <dijkstra.h>
```

Public Member Functions

- [Grafo](#) (int v)
Inicializa un grafo de n vértices con una matriz de adyacencia, por lo tanto, vacía.
- void [agregarArista](#) (int inicio, int destino, T peso, bool dirigido)
Inserta una arista ponderada que conecta a dos vértices del grafo.
- void [dijkstra](#) (int inicio)
Implementa el algoritmo de Dijkstra para calcular la distancia mínima desde un vértice de origen a todos los demás vértices del grafo.

9.1.1 Detailed Description

```
template<typename T>  
class Grafo< T >
```

Definition at line 20 of file [dijkstra.h](#).

9.1.2 Constructor & Destructor Documentation

9.1.2.1 Grafo()

```
template<typename T>  
Grafo< T >::Grafo (  
    int v)
```

Inicializa un grafo de n vértices con una matriz de adyacencia, por lo tanto, vacía.

A partir del número de vértices, se re-dimensiona un grafo del mismo tamaño cuya matriz de adyacencia (matriz↔_adj) empieza en ceros, lo cual indica que no hay aristas existentes entre los vertices.

Parameters

<i>v</i>	Número de vértices del grafo.
----------	-------------------------------

Definition at line 21 of file [dijkstra.cpp](#).

```
00022 {
00023     vertices = v;
00024     matriz_adj.resize(vertices, vector<T>(vertices, 0));
00025 }
```

9.1.3 Member Function Documentation**9.1.3.1 agregarArista()**

```
template<typename T>
void Grafo< T >::agregarArista (
    int inicio,
    int destino,
    T peso,
    bool dirigido)
```

Inserta una arista ponderada que conecta a dos vértices del grafo.

Se le asigna el peso de la arista a su posición respectiva en la matriz de adyacencia. Dado el caso que el grafo sea no dirigido, se asigna el mismo peso a la posición simétrica de la matriz de adyacencia.

Parameters

<i>inicio</i>	Vértice de inicio.
<i>destino</i>	Vértice de destino.
<i>peso</i>	Peso de la arista.
<i>dirigido</i>	Indica si es un grafo dirigido o no.

Definition at line 69 of file [dijkstra.cpp](#).

```
00070 {
00071     matriz_adj[inicio][destino] = peso;
00072     if (!dirigido)
00073     {
00074         matriz_adj[destino][inicio] = peso;
00075     }
00076 }
```

9.1.3.2 dijkstra()

```
template<typename T>
void Grafo< T >::dijkstra (
    int inicio)
```

Implementa el algoritmo de Dijkstra para calcular la distancia mínima desde un vértice de origen a todos los demás vértices del grafo.

Primero, se inicializa un vector de distancias con el valor máximo para el tipo de dato T, además de un vector de visitados en "false" para indicar que no han sido visitados. La distancia del vértice de inicio consigo mismo es de 0.

Se itera sobre los vértices del grafo, encontrado el vértice no visitado con la distancia mínima acumulada, marcándolo como visitado y actualizando las distancias de sus vértices adyacentes. Si se encuentra un camino más corto a través del vértice actual, se actualiza la distancia en el vector de distancias.

Parameters

<i>inicio</i>	Vértice de origen.
---------------	--------------------

< No hay vértices alcanzables.

Definition at line 93 of file [dijkstra.cpp](#).

```

00094 {
00095     vector<T> dist(vertices, numeric_limits<T>::max());
00096     vector<bool> visitado(vertices, false);
00097
00098     dist[inicio] = 0;
00099
00100     for (int i = 0; i < vertices - 1; i++)
00101     {
00102         int u = distanciaMin(dist, visitado);
00103
00104         if (u == -1)
00105         {
00106             break;
00107         }
00108
00109         visitado[u] = true;
00110
00111         for (int v = 0; v < vertices; v++)
00112         {
00113             if (!visitado[v] && matriz_adj[u][v] != 0 && dist[u] != numeric_limits<T>::max() &&
dist[u] + matriz_adj[u][v] < dist[v])
00114             {
00115                 dist[v] = dist[u] + matriz_adj[u][v];
00116             }
00117         }
00118     }
00119
00120     cout << "____ Distancias desde el vértice " << inicio << " ____\n\n";
00121     for (int i = 0; i < vertices; i++)
00122     {
00123         cout << "--> Para llegar al vertice " << i << " : " << dist[i] << endl;
00124     }
00125 }
```

The documentation for this class was generated from the following files:

- [headers/dijkstra.h](#)
- [src/dijkstra.cpp](#)

Chapter 10

File Documentation

10.1 headers/dijkstra.h File Reference

Se define la estructura de un grafo pesado para la implementación del algoritmo de Dijkstra.

```
#include <vector>
#include <limits>
#include "../src/dijkstra.cpp"
```

Classes

- class [Grafo< T >](#)

10.1.1 Detailed Description

Se define la estructura de un grafo pesado para la implementación del algoritmo de Dijkstra.

Author

Camila Andrea Jaimes Martínez.

Utilizando una matriz de adyacencia para representar el grafo, la plantilla de clase [Grafo](#) permite trabajar con diferentes tipos de peso, sean enteros o decimales. A partir de los vertices, el algoritmo de Dijkstra se especializa en calcular la distancia mínima desde un vértice "inicio a los demás vértices del grafo.

Definition in file [dijkstra.h](#).

10.2 dijkstra.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 #include <vector>
00003 #include <limits>
00004
00005 using namespace std;
00006
00017
00018
00019 template <typename T>
00020 class Grafo
00021 {
00022     private:
00023         int vertices;
00024         vector<vector<T>> matriz_adj;
00025
00026         int distanciaMin(vector<T>& dist, vector<bool>& visitado);
00027     public:
00028         Grafo(int v);
00029
00030         void agregarArista(int inicio, int destino, T peso, bool dirigido);
00031         void dijkstra(int inicio);
00032 };
00033
00034 #include "../src/dijkstra.cpp"
```

10.3 README.md File Reference

10.4 src/dijkstra.cpp File Reference

Implementación de de los constructores y métodos de la plantilla [Grafo](#).

```
#include <iostream>
#include "dijkstra.h"
```

10.4.1 Detailed Description

Implementación de de los constructores y métodos de la plantilla [Grafo](#).

Definition in file [dijkstra.cpp](#).

10.5 dijkstra.cpp

[Go to the documentation of this file.](#)

```
00001 #include <iostream>
00002 #include "dijkstra.h"
00003
00008
00009 using namespace std;
00010
00011 /*****
00012
00019
00020 template <typename T>
00021 Grafo<T>::Grafo(int v)
00022 {
00023     vertices = v;
00024     matriz_adj.resize(vertices, vector<T>(vertices, 0));
```

```

00025 }
00026
00027 /*****
00028
00037
00038 template <typename T>
00039 int Grafo<T>::distanciaMin(vector<T>& dist, vector<bool>& visitado)
00040 {
00041     T min = numeric_limits<T>::max();
00042     int indice_min = -1;
00043
00044     for (int i = 0; i < vertices; i++)
00045     {
00046         if (!visitado[i] && dist[i] < min)
00047         {
00048             min = dist[i];
00049             indice_min = i;
00050         }
00051     }
00052
00053     return indice_min;
00054 }
00055
00056 /*****
00057
00067
00068 template <typename T>
00069 void Grafo<T>::agregarArista(int inicio, int destino, T peso, bool dirigido)
00070 {
00071     matriz_adj[inicio][destino] = peso;
00072     if (!dirigido)
00073     {
00074         matriz_adj[destino][inicio] = peso;
00075     }
00076 }
00077
00078
00079 /*****
00080
00091
00092 template <typename T>
00093 void Grafo<T>::dijkstra(int inicio)
00094 {
00095     vector<T> dist(vertices, numeric_limits<T>::max());
00096     vector<bool> visitado(vertices, false);
00097
00098     dist[inicio] = 0;
00099
00100     for (int i = 0; i < vertices - 1; i++)
00101     {
00102         int u = distanciaMin(dist, visitado);
00103
00104         if (u == -1)
00105         {
00106             break;
00107         }
00108
00109         visitado[u] = true;
00110
00111         for (int v = 0; v < vertices; v++)
00112         {
00113             if (!visitado[v] && matriz_adj[u][v] != 0 && dist[u] != numeric_limits<T>::max() &&
dist[u] + matriz_adj[u][v] < dist[v])
00114             {
00115                 dist[v] = dist[u] + matriz_adj[u][v];
00116             }
00117         }
00118     }
00119
00120     cout << "----- Distancias desde el vértice " << inicio << " -----\\n\\n";
00121     for (int i = 0; i < vertices; i++)
00122     {
00123         cout << "--> Para llegar al vertice " << i << " : " << dist[i] << endl;
00124     }
00125 }

```

10.6 src/main.cpp File Reference

El programa completo de la clase [Grafo](#) y el algoritmo de Dijkstra.

```
#include <iostream>
#include "dijkstra.h"
```

Functions

- int [menu](#) ()
Menú del programa que maneja el input (aportación) del usuario.
- int [main](#) ()

10.6.1 Detailed Description

El programa completo de la clase [Grafo](#) y el algoritmo de Dijkstra.

Las opciones son las siguientes:

- 1) Crear un grafo con un número determinado de vértices.
- 2) Agregar aristas ponderadas, dirigidas o no, entre dos vértices del grafo.
- 3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los demás.
- 4) Salir del programa.

Definition in file [main.cpp](#).

10.6.2 Function Documentation

10.6.2.1 main()

```
int main ()
```

< Número de vértices del grafo.

< [Grafo](#) con n vertices y pesos enteros.

< Elimina el grafo anterior si fue creado previamente.

< Vértice de inicio, de destino, y peso de la arista, respectivamente.

< Indica si el grafo es dirigido o no.

< Vértice de inicio para el algoritmo de Dijkstra.

< Libera la memoria del grafo antes de salir del programa.

Definition at line 19 of file [main.cpp](#).

```
00020 {
00021     int n;
00022     Grafo<int>* grafo = nullptr;
00023
00024     while (true)
00025     {
00026         switch (menu())
00027         {
```

```

00028         case 1:
00029         {
00030             cout << "\n¿De cuántos vértices desea crear el grafo?: ";
00031             cin >> n;
00032
00033             if (grafo != nullptr)
00034             {
00035                 delete grafo;
00036             }
00037
00038             grafo = new Grafo<int>(n);
00039             cout << "\n¡El grafo se ha creado exitosamente con " << n << " vértices!";
00040             break;
00041         }
00042         case 2:
00043         {
00044             if (grafo == nullptr)
00045             {
00046                 cout << "\nNo se ha creado un grafo aún.";
00047                 break;
00048             }
00049
00050             int in, dest, p;
00051             bool direccion;
00052
00053             cout << "\nIngrese el vértice de inicio: ";
00054             cin >> in;
00055
00056             cout << "\nIngrese el vértice de destino: ";
00057             cin >> dest;
00058
00059             cout << "\nIngrese el peso de la arista: ";
00060             cin >> p;
00061
00062             cout << "\n¿Es un grafo dirigido? (1 para sí, 0 para no): ";
00063             cin >> direccion;
00064
00065             grafo->agregarArista(in, dest, p, direccion);
00066             cout << "\n¡La arista se ha creado exitosamente!";
00067             break;
00068         }
00069         case 3:
00070         {
00071             if (grafo == nullptr)
00072             {
00073                 cout << "\nNo se ha creado un grafo aún.";
00074                 break;
00075             }
00076
00077             int origen;
00078
00079             cout << "\n¿Cuál es el vértice de inicio? (Implementación del algoritmo de Dijkstra): ";
00080             cin >> origen;
00081
00082             grafo->dijkstra(origen);
00083             break;
00084         }
00085         case 4:
00086         {
00087             cout << "\nSaliendo del programa...";
00088             exit(0);
00089             break;
00090         }
00091     }
00092 }
00093
00094 delete grafo;
00095 }

```

10.6.2.2 menu()

```
int menu ()
```

Menú del programa que maneja el input (aportación) del usuario.

Devuelve una opción a la función [main\(\)](#).

Definition at line 105 of file [main.cpp](#).

```
00106 {
```

```

00107     int opcion;
00108     do
00109     {
00110         cout << "\n\n----- ; MENÚ ! -----";
00111         cout << "\n\n1) Crear un grafo de n vértices.\n";
00112         cout << "2) Agregar aristas ponderadas.\n";
00113         cout << "3) Calcular y visualizar la distancia mínima desde un vértice de origen a todos los
demás.\n";
00114         cout << "4) Salir del programa.\n";
00115         cout << "\n--> Seleccione una opción: ";
00116         cin >> opcion;
00117     } while (opcion < 1 || opcion > 4);
00118     return opcion;
00119 }

```

10.7 main.cpp

[Go to the documentation of this file.](#)

```

00001 #include <iostream>
00002 #include "dijkstra.h"
00003
00004
00005 int menu();
00006
00007 using namespace std;
00008
00009 int main()
00010 {
00011     int n;
00012     Grafo<int>* grafo = nullptr;
00013
00014     while (true)
00015     {
00016         switch (menu())
00017         {
00018             case 1:
00019             {
00020                 cout << "\n¿De cuántos vértices desea crear el grafo?: ";
00021                 cin >> n;
00022
00023                 if (grafo != nullptr)
00024                 {
00025                     delete grafo;
00026                 }
00027
00028                 grafo = new Grafo<int>(n);
00029                 cout << "\nEl grafo se ha creado exitosamente con " << n << " vértices!";
00030                 break;
00031             }
00032             case 2:
00033             {
00034                 if (grafo == nullptr)
00035                 {
00036                     cout << "\nNo se ha creado un grafo aún.";
00037                     break;
00038                 }
00039
00040                 int in, dest, p;
00041                 bool direccion;
00042
00043                 cout << "\nIngrese el vértice de inicio: ";
00044                 cin >> in;
00045
00046                 cout << "\nIngrese el vértice de destino: ";
00047                 cin >> dest;
00048
00049                 cout << "\nIngrese el peso de la arista: ";
00050                 cin >> p;
00051
00052                 cout << "\n¿Es un grafo dirigido? (1 para sí, 0 para no): ";
00053                 cin >> direccion;
00054
00055                 grafo->agregarArista(in, dest, p, direccion);
00056                 cout << "\nLa arista se ha creado exitosamente!";
00057                 break;
00058             }
00059             case 3:
00060             {
00061                 if (grafo == nullptr)
00062                 {
00063                     cout << "\nNo se ha creado un grafo aún.";
00064                 }
00065             }
00066         }
00067     }
00068 }

```

```

00074         break;
00075     }
00076
00077     int origen;
00078
00079     cout << "\n¿Cuál es el vértice de inicio? (Implementación del algoritmo de Dijkstra): ";
00080     cin >> origen;
00081
00082     grafo->dijkstra(origen);
00083     break;
00084 }
00085 case 4:
00086 {
00087     cout << "\nSaliendo del programa...";
00088     exit(0);
00089     break;
00090 }
00091 }
00092 }
00093
00094 delete grafo;
00095 }
00096
00097 /*****
00098
00104
00105 int menu()
00106 {
00107     int opcion;
00108     do
00109     {
00110         cout << "\n\n----- ; MENÚ ! -----";
00111         cout << "\n\n1) Crear un grafo de n vértices.\n";
00112         cout << "2) Agregar aristar ponderadas.\n";
00113         cout << "3) Calcular y visualzar la distancia mínima desde un vértice de origen a todos los
00114 demás.\n";
00115         cout << "4) Salir del programa.\n";
00116         cout << "\n--> Seleccione una opción: ";
00117         cin >> opcion;
00118     } while (opcion < 1 || opcion > 4);
00119     return opcion;
00120 }
00121 /*****
00122
00165
00166 /*****
00167
00211
00212 /*****
00213

```


Index

agregarArista

 Grafo< T >, [26](#)

Análisis, [3](#)

Bibliografía, [15](#)

Determinación de circuitos en un grafo (C++), [1](#)

dijkstra

 Grafo< T >, [26](#)

Grafo

 Grafo< T >, [25](#)

Grafo< T >, [25](#)

 agregarArista, [26](#)

 dijkstra, [26](#)

 Grafo, [25](#)

headers Directory Reference, [23](#)

headers/dijkstra.h, [29](#), [30](#)

main

 main.cpp, [32](#)

main.cpp

 main, [32](#)

 menu, [33](#)

menu

 main.cpp, [33](#)

Pruebas, [5](#)

README.md, [30](#)

src Directory Reference, [23](#)

src/dijkstra.cpp, [30](#)

src/main.cpp, [31](#), [34](#)